

FLUTTER · BEGINNER-FRIENDLY GUIDE

Riverpod + Clean Architecture

Learn how state management & clean architecture work together — explained with real code from the *Bookly* project.

This guide is written for your **first time** with both Riverpod and Clean Architecture. It starts from zero concepts, then walks through the actual `auth` feature in your app, layer by layer, until you can add a new feature on your own.

Project: **bookly** · Prepared 19 July 2026

Packages used: **flutter_riverpod**, **fpdart** (Either), **equatable**, **dio**, **go_router**

What's inside

1. Part 1 — The big picture (why bother?)
2. Part 2 — Riverpod in plain words
3. Part 3 — Clean Architecture in plain words
4. Part 4 — How Bookly is organized
5. Part 5 — Layer by layer with YOUR code (Domain → Data → Presentation)
6. Part 6 — The Riverpod provider cheat-sheet
7. Part 7 — read vs watch vs listen
8. Part 8 — Full end-to-end flow: what happens when you tap "Sign In"
9. Part 9 — Recipe: adding a brand-new feature
10. Part 10 — Common mistakes & tips
11. Part 11 — Glossary

Part 1 — The big picture (why bother?)

Two separate ideas are combined in your app. It helps to keep them apart in your head:

Idea	Question it answers	In Bookly
Clean Architecture	"Where does each piece of code live, and who is allowed to talk to whom?"	The <code>domain / data / presentation</code> folders inside every feature.
Riverpod	"How do I store state, share it, and rebuild the UI when it changes?"	The <code>*_provider.dart</code> files + <code>ProviderScope</code> + <code>ConsumerWidget</code> .

One-line summary: Clean Architecture decides the **folders and boundaries**. Riverpod is the **wiring** that carries data across those boundaries into your screens.

Why not just put everything in the widget?

You *can* — for a tiny app. But as the app grows you get:

- UI code tangled with network calls → hard to read, hard to test.
- The same logic copy-pasted in 5 screens → change it in one place, forget the other four.
- Swapping the backend (e.g. REST → Firebase) means editing every screen.

Clean Architecture + Riverpod fixes this by giving every piece of code **one job** and **one home**.

Part 2 — Riverpod in plain words

Riverpod is a **state management** library. Three ideas are all you need to start:

1. A "provider" is a box that holds something

A provider is a global variable that **creates and holds** a value — a service, a piece of state, a stream, anything. You declare it once at the top level:

```
// A box that holds an AuthRepository object.
final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepositoryImpl();
});
```

The `ref` handed to you is your key to **every other box** — that's how boxes are wired together.

2. Widgets "watch" a box to get its value (and rebuild when it changes)

To read a provider from a widget, the widget must be a `ConsumerWidget` (or `ConsumerStatefulWidget`), which gives you a `ref`:

```
final isLoading = ref.watch(authControllerProvider);
// isLoading is now a bool. If the box's value changes,
// this widget automatically rebuilds. No setState needed.
```

3. One shared source of truth

Because a provider is global, screen A and screen B watching the same provider always see the **exact same value**. Update it in one place → both screens update. No passing data down through constructors.

The whole app must be wrapped in a `ProviderScope` for any of this to work. In Bookly that happens once, at the very top:

```
// lib/src/shared/wrappers/state_wrapper.dart
class StateWrapper extends StatelessWidget {
  final Widget child;
  const StateWrapper({super.key, required this.child});

  @override
  Widget build(BuildContext context) {
    return ProviderScope(child: child); // <-- Riverpod turned on here
  }
}
```

```
// lib/main.dart - StateWrapper wraps the whole App  
runApp(  
  const StateWrapper(  
    child: App(),  
  ),  
);
```

Part 3 — Clean Architecture in plain words

Clean Architecture splits a feature into **three layers**. Think of them as three shelves, where a higher shelf may reach *down* but a lower shelf must never reach *up*.

Presentation layer — the screens & state

Everything the user sees and interacts with: widgets, screens, and the Riverpod providers / controllers that hold UI state.

In Bookly: `presentation/screens` and `presentation/providers` .

▼ depends on ▼

Domain layer — the pure rules (the heart)

The business rules, written in plain Dart with **no Flutter, no Dio, no packages**. It contains **Entities** (plain objects like `AppUser`) and **Repository interfaces** (abstract contracts describing *what* can be done, not *how*).

In Bookly: `domain/entities` and `domain/repositories` .

▲ implemented by ▲

Data layer — the real implementation

The concrete code that actually talks to the network / database: API calls (Dio), JSON parsing (Models), and the class that fulfils the domain's repository contract.

In Bookly: `data/models` and `data/repositories` .

The one rule that matters: the Dependency Rule

Arrows always point INWARD, toward the Domain.

Presentation knows about Domain. Data knows about Domain. But **Domain knows about nobody** — it never imports Presentation or Data. This is what keeps the "heart" of your app clean, testable, and independent of Flutter or your backend.

The magic trick that makes this possible is the **abstract repository**: the Domain declares an interface, the Data layer implements it, and Presentation only ever talks to the interface. So the Domain never needs to know Dio even exists.

Part 4 — How Bookly is organized

Every feature lives in `lib/src/features/<feature>/` and repeats the same three-layer shape. Here is the real `auth` feature:

```
lib/src/features/auth/
├── domain/                               ← pure rules, no packages
│   ├── entities/
│   │   └── user.dart                     AppUser (plain object)
│   └── repositories/
│       └── auth_repository.dart         AuthRepository (abstract contract)
├── data/                                ← real implementation
│   ├── models/
│   │   └── user_model.dart              maps JSON ↔ AppUser
│   └── repositories/
│       └── auth_repository_impl.dart    implements AuthRepository
└── presentation/                        ← screens + Riverpod
    ├── providers/
    │   ├── auth_provider.dart           AuthController + providers
    │   └── session_provider.dart        who is logged in
    └── screens/
        ├── login_screen.dart
        ├── signup_screen.dart
        └── forgot_password_screen.dart
```

Notice the pattern. The three folders (`domain` , `data` , `presentation`) are exactly the three layers from Part 3. Every new feature you build will copy this same skeleton.

Supporting pieces shared by all features

File	What it gives you
<code>utils/typedefs.dart</code>	<code>FutureEither<T></code> — the return type for anything that can fail.
<code>utils/failure.dart</code>	<code>Failure</code> types (<code>ServerFailure</code> , <code>NetworkFailure</code> ...).
<code>utils/task_runner.dart</code>	<code>runTask()</code> — wraps risky calls in try/catch and returns an <code>Either</code> .
<code>services/</code>	Thin wrappers around raw tools (Dio, storage, location...). Used by the Data layer.

Part 5 — Layer by layer with YOUR code

Now the fun part. We'll read the real `auth` feature from the bottom of the dependency chain (Domain) up to the top (Presentation) — the same order you should build in.

5.1 Domain — the Entity

domain/entities/user.dart

```
import 'package:equatable/equatable.dart';

class AppUser extends Equatable {
  final String id;
  final String email;
  final String? name;
  final String? photoUrl;

  const AppUser({
    required this.id,
    required this.email,
    this.name,
    this.photoUrl,
  });

  factory AppUser.empty() => const AppUser(id: '', email: '');

  bool get isEmpty => id.isEmpty;
  bool get isNotEmpty => id.isNotEmpty;

  @override
  List<Object?> get props => [id, email, name, photoUrl];
}
```

- **Pure Dart.** The only import is `equatable` — no Flutter, no Dio. This is the Domain rule in action.
- `extends Equatable` + `props` means two `AppUser` s with the same fields are considered **equal**. This lets Riverpod know when state *really* changed and avoid useless rebuilds.
- An **Entity** represents a real business concept ("a user"), independent of how it's stored or shown.

5.2 Domain — the Repository contract

domain/repositories/auth_repository.dart


```
abstract class AuthRepository {
  /// Emits AppUser when authenticated, null when not.
  Stream<AppUser?> get onAuthStateChanged;

  FutureEither<AppUser> login({required String email, required String password});

  FutureEither<AppUser> signUp({required String name, required String email, required String password});

  FutureEither<void> forgotPassword({required String email});

  FutureEither<void> logout();

  FutureEither<AppUser?> checkAuthState();
}
```

This is the most important idea in the whole app. This file is a **contract**: it lists *what* auth can do, but contains **zero** implementation. It's **abstract**. The Presentation layer will depend only on this contract, never on the real network code — which is exactly what lets you swap backends or write fake versions for tests.

What is `FutureEither<T>` ?

`utils/typedefs.dart`

```
typedef FutureEither<T> = Future<Either<Failure, T>>;
```

An `Either<Failure, T>` (from the `fpdart` package) holds **one of two things**: a `Failure` on the **Left**, or a success value `T` on the **Right**. So `FutureEither<AppUser>` means: "an async operation that will finish with **either** a Failure **or** an AppUser." This forces you to handle errors — you can't accidentally forget them like you can with exceptions.

5.3 Data — the Service (raw backend calls)

Before the repository implementation, there's a thin **Service** that does the raw HTTP work with Dio. It knows nothing about your Domain entities — it just returns raw maps.

services/auth_service.dart (excerpt)

```
class AuthService {
  AuthService._();
  static final AuthService instance = AuthService._(); // singleton

  Dio get _dio => AppConfig.dio;

  FutureEither<Map<String, dynamic>?> login({
    required String email,
    required String password,
  }) async {
    return runTask(() async {
      final response = await _dio.post('/auth/login', data: {
        'email': email,
        'password': password,
      });
      final data = response.data!;
      _authStateController.add(data); // notify the auth stream
      return data;
    }, requiresNetwork: true);
  }
}
```

The `runTask(...)` helper (in `utils/task_runner.dart`) wraps the call in try/catch, checks the network, and turns any exception into a `Left(ServerFailure)`. That's how a raw `throw` becomes a safe `Either`.

5.4 Data — the Repository implementation

data/repositories/auth_repository_impl.dart (excerpt)

```

// Note: it IMPLEMENTS the domain contract.
class AuthRepositoryImpl implements AuthRepository {
    final AuthService _authService = AuthService.instance;

    @override
    FutureEither<AppUser> login({
        required String email,
        required String password,
    }) async {
        final result = await _authService.login(email: email, password: password);

        // result is Either<Failure, Map>. flatMap runs only on success.
        return result.flatMap((userData) {
            if (userData == null) {
                return left(const ServerFailure('Login failed: User record not found'));
            }
            final data = userData['user'] ?? userData;
            // Convert raw JSON map -> a clean Domain entity
            final user = AppUser(
                id: data['id'].toString(),
                email: data['email'] ?? email,
                name: data['name'],
            );
            return right(user);
        });
    }
}

```

This class is the bridge. It takes messy JSON from the backend and returns a clean `AppUser` entity that the rest of the app understands. This is the **only** place that knows both worlds (raw JSON *and* the Domain). Presentation never sees a JSON map.

5.5 Presentation — wiring the repository into Riverpod

Now we're in the Riverpod world. First, expose the repository as a provider so screens can reach it. This is the **seam** where the interface meets its implementation:

presentation/providers/auth_provider.dart

```
// The box returns the CONTRACT type (AuthRepository)...  
// ...but builds the CONCRETE class (AuthRepositoryImpl).  
final authRepositoryProvider = Provider<AuthRepository>((ref) {  
  return AuthRepositoryImpl();  
});
```

Why is this powerful? Everything else asks for `AuthRepository` (the interface). Only **this single line** decides which real class fulfils it. To use a fake repo in a test, you override just this one provider — nothing else in the app changes.

5.6 Presentation — the Controller (StateNotifier)

A **controller** holds UI state and runs the actions when the user taps a button. Here the state is a simple `bool` meaning "is a request in progress?".

presentation/providers/auth_provider.dart (continued)

```
// StateNotifierProvider exposes a controller + its state (bool).
final authControllerProvider =
  StateNotifierProvider<AuthController, bool>((ref) {
    return AuthController(
      repository: ref.read(authRepositoryProvider), // grab the repo from its box
    );
  });

class AuthController extends StateNotifier<bool> {
  final AuthRepository _repository; // depends on the INTERFACE only

  AuthController({required AuthRepository repository})
    : _repository = repository,
      super(false); // initial state: not loading

  void login({
    required BuildContext context,
    required String email,
    required String password,
  }) async {
    state = true; // start loading -> UI shows spinner automatically

    final result = await _repository.login(email: email, password: password);

    state = false; // stop loading
    result.fold(
      (failure) { // LEFT branch = error
        if (context.mounted) {
          showToast(context, message: failure.message, status: 'error');
        }
      },
      (user) { // RIGHT branch = success
        if (rootContext?.mounted ?? false) {
          rootContext!.go(AppRoutes.home);
        }
      },
    );
  }
}
```

- `state = true/false` — assigning to `state` is what makes watching widgets rebuild.
- `result.fold(onError, onSuccess)` — the clean way to handle both branches of an `Either`. You are **forced** to handle the error case.
- The controller talks to `_repository` (the interface). It has **no idea** Dio exists. That's the dependency rule working for you.

5.7 Presentation — the Screen consuming it all

Finally the UI. The screen is a `ConsumerStatefulWidget` so it gets a `ref`.

presentation/screens/login_screen.dart (excerpt)

```
class LoginScreen extends ConsumerStatefulWidget {
  const LoginScreen({super.key});
  @override
  ConsumerState<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends ConsumerState<LoginScreen> {
  @override
  Widget build(BuildContext context) {
    // WATCH the loading state -> rebuild whenever it flips.
    final isLoading = ref.watch(authControllerProvider);

    Future<void> handleLogin() async {
      if (!(_formKey.currentState?.validate() ?? false)) return;

      // READ the controller (.notifier) and CALL its action.
      ref.read(authControllerProvider.notifier).login(
        context: context,
        email: _emailController.text,
        password: _passwordController.text,
      );
    }

    return _LoginView(
      isLoading: isLoading,           // disable inputs / show spinner
      onLogin: handleLogin,
      /* ...other props... */
    );
  }
}
```

The two-move pattern you'll use everywhere:

- `ref.watch(provider)` to **read state** and rebuild on change (the value).
- `ref.read(provider.notifier)` to **call an action** (the controller). Use `.read` inside callbacks like `onPressed`, never `.watch`.

Part 6 — The Riverpod provider cheat-sheet

Bookly uses these provider types. Pick by **what kind of value** you're holding:

Provider type	Use it when...	Example in Bookly
<code>Provider</code>	You expose a value/object that doesn't change (a service, a repository).	<code>authRepositoryProvider</code>
<code>StateNotifierProvider</code>	You have changing state + logic that mutates it. The workhorse for forms & actions.	<code>authControllerProvider</code> , <code>sessionProvider</code>
<code>StreamProvider</code>	Your value arrives over time from a Stream (e.g. live auth state).	<code>authStateStreamProvider</code>
<code>FutureProvider</code>	You fetch once asynchronously and want auto loading/error handling (<code>AsyncValue</code>).	(pattern you'll add for lists, e.g. fetching books)
<code>StateProvider</code>	A single, simple mutable value (a toggle, a selected tab index) with no logic class.	(handy for small UI state)

The session provider — a richer `StateNotifier` example

Unlike the loading-bool controller, `SessionNotifier` holds a **custom state class**. This is the normal shape once your state has more than one field:

```
presentation/providers/session_provider.dart (excerpt)
```

```

enum SessionStatus { unknown, authenticated, unauthenticated }

class SessionState {
    final SessionStatus status;
    final AppUser? user;
    const SessionState({this.status = SessionStatus.unknown, this.user});

    // copyWith = make a changed copy without mutating the original.
    SessionState copyWith({SessionStatus? status, AppUser? user}) {
        return SessionState(
            status: status ?? this.status,
            user: user ?? this.user,
        );
    }
}

class SessionNotifier extends StateNotifier<SessionState> {
    final AuthRepository _repository;

    SessionNotifier({required AuthRepository repository})
        : _repository = repository,
          super(const SessionState()) {
        _init(); // check persisted login as soon as it's created
    }
    // ...listens to _repository.onAuthStateChanged and updates state...
}

```

Note it also uses a `StreamProvider` nearby: `authStateStreamProvider = StreamProvider<AppUser?>((ref) => ref.watch(authRepositoryProvider).onAuthStateChanged);`

Part 7 — read vs watch vs listen

These three are the source of most beginner confusion. Learn them once and you're set.

Method	What it does	Where to use it
<code>ref.watch(p)</code>	Reads the value and subscribes . The widget/provider rebuilds whenever <code>p</code> changes.	Inside <code>build()</code> , to get values you want to display.
<code>ref.read(p)</code>	Reads the value once , no subscription, no rebuild.	Inside callbacks (<code>onPressed</code> , <code>onTap</code>) — usually with <code>.notifier</code> to call an action.
<code>ref.listen(p, cb)</code>	Runs a side-effect when <code>p</code> changes (navigate, show a toast) — without rebuilding.	Inside <code>build()</code> , for one-off reactions to state changes.

`ref.listen` in your app — the session redirect

`shared/wrappers/session_listener_wrapper.dart`

```
class SessionListenerWrapper extends ConsumerWidget {
  final Widget child;
  const SessionListenerWrapper({super.key, required this.child});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // React to login/logout by navigating – a side effect, not a rebuild.
    ref.listen<SessionState>(sessionProvider, (prev, next) {
      if (next.status != SessionStatus.unknown) {
        FlutterNativeSplash.remove();
        if (next.status == SessionStatus.authenticated) {
          context.go(AppRoutes.home);
        } else if (next.status == SessionStatus.unauthenticated) {
          context.go(AppRoutes.onboarding);
        }
      }
    });
    return child;
  }
}
```

Golden rule: never call `ref.watch` inside a button callback (it can trigger endless rebuilds), and never navigate/show dialogs from inside `build()` directly — use `ref.listen` for that.

Part 8 — Full end-to-end flow: tapping "Sign In"

This ties everything together. Follow the data as it crosses each layer and comes back.

- 1 User taps the button.** `LoginScreen` validates the form, then calls `ref.read(authControllerProvider.notifier).login(...)`. (Presentation layer)
- 2 Controller sets loading.** `AuthController.login` does `state = true`. The screen is watching, so the button shows a spinner instantly. (Presentation)
- 3 Controller calls the contract.** It awaits `_repository.login(email, password)` — but `_repository` is just the `AuthRepository` interface. (Presentation → Domain contract)
- 4 Implementation runs.** Riverpod wired the real `AuthRepositoryImpl` in, so its `login` executes and calls `AuthService.login`. (Data layer)
- 5 Network call.** `AuthService` uses Dio to POST `/auth/login`, wrapped in `runTask()` which returns `Right(json)` or `Left(Failure)`. (Data → network)
- 6 JSON → Entity.** Back in `AuthRepositoryImpl`, `flatMap` converts the raw map into a clean `AppUser` entity, returning `Right(user)`. (Data → Domain entity)
- 7 Controller reacts.** `state = false` (spinner off), then `result.fold(onError, onSuccess)`: on error show a toast; on success navigate to Home. (Presentation)
- 8 Session updates too.** The service also pushed the new user into its auth stream, so `SessionNotifier` hears it and `SessionListenerWrapper` redirects accordingly. (Presentation, via Stream)

Read it top to bottom: Presentation → Domain contract → Data → back up. The Domain never imported anything below it, yet everything flowed through it. That's Clean Architecture doing its job.

Part 9 — Recipe: adding a brand-new feature

Say you want a **Books** feature that fetches a list of books. Copy the `auth` skeleton and fill in these steps **in this order** (Domain first, UI last):

Step 1 — Domain: the entity

features/books/domain/entities/book.dart

```
class Book extends Equatable {
  final String id;
  final String title;
  final String author;
  const Book({required this.id, required this.title, required this.author});
  @override List<Object?> get props => [id, title, author];
}
```

Step 2 — Domain: the contract

features/books/domain/repositories/book_repository.dart

```
abstract class BookRepository {
  FutureEither<List<Book>> getBooks();
}
```

Step 3 — Data: implement it

features/books/data/repositories/book_repository_impl.dart

```
class BookRepositoryImpl implements BookRepository {
  final BookService _service = BookService.instance;

  @override
  FutureEither<List<Book>> getBooks() async {
    final result = await _service.fetchBooks(); // Either<Failure, List<Map>>
    return result.map((list) => list
      .map((json) => Book(
        id: json['id'].toString(),
        title: json['title'] ?? '',
        author: json['author'] ?? '',
      ))
      .toList());
  }
}
```

Step 4 — Presentation: providers

features/books/presentation/providers/book_provider.dart

```
final bookRepositoryProvider = Provider<BookRepository>((ref) {
    return BookRepositoryImpl();
});

// FutureProvider is perfect for a one-shot fetch: it gives you
// loading / data / error handling for free via AsyncValue.
final booksProvider = FutureProvider<List<Book>>((ref) async {
    final repo = ref.watch(bookRepositoryProvider);
    final result = await repo.getBooks();
    return result.fold((f) => throw f, (books) => books);
});
```

Step 5 — Presentation: the screen

```
class BooksScreen extends ConsumerWidget {
    const BooksScreen({super.key});
    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final booksAsync = ref.watch(booksProvider);

        // AsyncValue handles all three states for you.
        return booksAsync.when(
            loading: () => const AppLoading(),
            error: (e, _) => AppErrorWidget(message: '$e'),
            data: (books) => ListView(
                children: books.map((b) => ListTile(title: Text(b.title))).toList(),
            ),
        );
    }
}
```

That's the whole recipe. Entity → Contract → Impl → Providers → Screen. Notice you built from the **inside out** (Domain first), which matches the dependency rule.

Part 10 — Common mistakes & tips

1. Using `ref.watch` in a callback. In `onPressed` use `ref.read`. `watch` is only for `build()`.

2. Importing Data or Flutter inside Domain. If your entity or repository interface imports `dio`, `flutter/material`, or anything from `data/`, you broke the dependency rule. Domain stays pure.

3. Returning raw JSON to the UI. Convert maps into entities inside the repository *impl*. Screens should only ever see entities like `AppUser`.

4. Forgetting `ProviderScope`. If a provider "isn't found", check the app is wrapped in `ProviderScope` (Bookly does this in `StateWrapper`).

5. Mutating state in place. With a custom state class, always create a new object (use `copyWith`). Riverpod detects change by comparing objects; mutating the old one may not trigger a rebuild.

6. Let `Either` force you to handle errors. Always `.fold` (or `.when` for `AsyncValue`). The types are nudging you toward safe code — lean into it.

7. Build inside-out. When adding a feature, write Domain → Data → Presentation. It feels backwards at first but prevents rework.

8. Keep the controller thin. Its job is: set loading, call the repository, react. Business logic that transforms data belongs in the Data/Domain layer, not the UI.

Part 11 — Glossary

Term	Meaning in this project
Entity	A plain business object (e.g. <code>AppUser</code>), pure Dart, lives in Domain.
Model	Data-layer version that knows how to parse JSON. Converts to/from an Entity.
Repository (interface)	An <code>abstract class</code> in Domain listing what a feature can do.
Repository Impl	The Data-layer class that actually does it (talks to services).
Service	Thin wrapper over a raw tool (Dio, storage). Returns raw data.
Provider	A Riverpod "box" that creates and shares a value app-wide.
StateNotifier	A class holding mutable <code>state</code> + methods that change it.
Controller	In Bookly, a StateNotifier that drives a screen's actions (e.g. <code>AuthController</code>).
ref	Your handle to read/watch/listen other providers.
ConsumerWidget	A widget that receives a <code>ref</code> so it can use providers.
Either<L,R>	Holds success (Right) OR failure (Left). From <code>fpdart</code> .
FutureEither<T>	<code>Future<Either<Failure, T>></code> — an async op that can fail.
AsyncValue	Wraps loading / data / error for <code>Future</code> / <code>Stream</code> providers; use <code>.when(...)</code> .
fold / when	How you branch: <code>fold</code> for Either, <code>when</code> for AsyncValue.
Dependency Rule	Outer layers depend on inner; Domain depends on nothing.

Your one-page mental model

Screen watches a **Controller (StateNotifier)** for state and calls its actions → the Controller talks to a **Repository interface** (Domain) → Riverpod supplies the real **Repository Impl** (Data) → which calls a **Service** (Dio) and maps JSON into an **Entity** → the result (`Either`) flows back and the Controller updates `state` → the Screen rebuilds. Domain sits in the middle and depends on no one.

Generated from the real `bookly` source — auth feature. Keep this next to the code and trace each file as you read. You'll internalize the pattern in a day or two of building.